

中山大学计算机学院

人工智能

本科生实验报告

(2022 学年春季学期)

课程名称: Artificial Intelligence

教学班级	20240574	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇

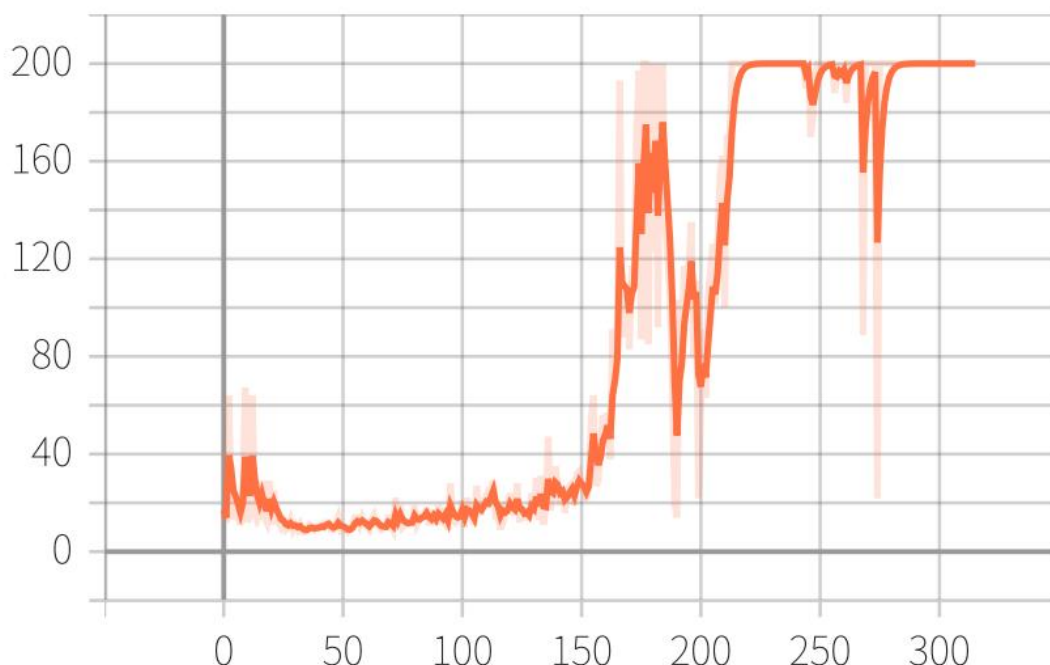
一、 实验题目

实现 DQN 算法

在 CartPole-v0 环境中实现 DQN 算法。最终算法性能的评判标准：以算法收敛的 reward 大小、收敛所需的样本数量给分。reward 越高（至少是 180，最大是 200）、收敛所需样本数量越少，分数越高。

作业提交内容：需提交一个 zip 文件，包括代码以及实验报告 PDF。实验报告除了需要写 writing 部分的内容，还需要给出 reward 曲线图以及算法细节。

我们给出 DQN 在 cartpole 环境的训练曲线图作为参考。



二、 实验内容

1. 算法原理

DQN 是一种结合了深度学习和强化学习的算法，用于解决高维状态空间下的决策问题。其核心思想是用深度神经网络（QNetwork）来逼近 Q-learning 中的动作价值函数 $Q(s, a)$ 。主要原理如下：

1.1 Q-learning 基础

Q-learning 通过不断更新 $Q(s, a)$ 来学习最优策略，更新公式为：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

其中 α 为学习率， γ 为折扣因子， r 为即时奖励。

1.2 深度神经网络逼近 Q 函数

由于状态空间过大，无法用表格存储 $Q(s, a)$ ，DQN 用神经网络 QNetwork 以状态为输入，输出所有动作的 Q 值。

1.3 经验回放（Replay Buffer）

为了打破数据相关性，DQN 采用经验回放池（ReplayBuffer），每次从池中随机采样一批数据训练网络，提高训练稳定性。

1.4 目标网络（Target Network）

为了避免训练过程中的目标值震荡，DQN 维护一个目标 Q 网络（target_q_net），其参数定期从主 Q 网络（q_net）同步。

1.5 epsilon-greedy 策略

训练时采用 epsilon-greedy 策略进行探索：以 ϵ 概率随机选择动作，其余时间选择 Q 值最大的动作。

2. DQN 算法在本项目中的体现

2.1 QNetwork 类

用于实现 Q 函数的神经网络逼近（QNetwork），输入为状态，输出为每个动作的 Q 值。

2.2 ReplayBuffer 类

实现经验回放机制，存储智能体与环境交互的 $(s, a, r, s', done)$ 经验，并支持随机采样。

2.3 AgentDQN 类

self.q_net 和 self.target_q_net 分别为主 Q 网络和目标 Q 网络。

self.buffer 为经验回放池。

self.epsilon 及其衰减实现 epsilon-greedy 策略。

train() 方法实现了 DQN 的核心训练逻辑，包括采样、计算目标 Q 值、损失函数、反向传播和目标网络同步。

make_action() 方法实现 epsilon-greedy 策略选择动作。

run() 方法为主循环，负责与环境交互、存储经验、训练网络和记录日志。



3. 伪代码(模板中的 agent_dqn.py)

1. 定义 QNetwork 神经网络类
 - 两层全连接网络, 实现 $Q(s, a)$ 估计
2. 定义经验回放池 ReplayBuffer
 - 存储和采样智能体与环境交互的经验
3. 定义 AgentDQN 智能体类, 继承 Agent
 - 初始化:
 - 构建 Q 网络和目标 Q 网络
 - 初始化优化器、经验池、超参数等
 - init_game_setting:
 - 每局游戏开始时重置探索率
 - train:
 - 若经验池数据不足, 返回
 - 从经验池采样一批数据
 - 计算当前 Q 值和目标 Q 值
 - 计算损失并反向传播, 更新主 Q 网络
 - 定期同步目标 Q 网络参数
 - make_action:
 - 按 epsilon-greedy 策略选择动作
 - epsilon 概率随机动作, 否则选择 Q 值最大的动作
 - run:
 - 多回合训练主循环
 - 每回合重置环境
 - 直到 done:
 - 选择动作并与环境交互
 - 存储经验
 - 累计奖励
 - 训练网络
 - 衰减 epsilon
 - 同步目标网络
 - 记录奖励到 TensorBoard 并打印信息

4. 关键代码展示（带注释）

4.1 Q 网络 QNetwork 类

```
class QNetwork(nn.Module): #定义Q网络，继承自 PyTorch 的 nn.Module

    def __init__(self, input_size, hidden_size, output_size):

        super(QNetwork, self).__init__() # 调用父类构造函数

        self.fc1 = nn.Linear(input_size, hidden_size) # 第一层全连接层，输入为状态维度，输出为隐藏层维度
        self.fc2 = nn.Linear(hidden_size, output_size) # 第二层全连接层，输入为隐藏层维度，输出为动作数

    def forward(self, inputs): # 前向传播函数

        x = torch.relu(self.fc1(inputs)) # 输入经过第一层并激活

        x = self.fc2(x) # 经过第二层输出Q值

        return x # 返回Q值
```

4.2 经验池 ReplayBuffer 类

```
class ReplayBuffer: # 定义经验回放池

    def __init__(self, buffer_size):

        self.buffer = [] # 用于存储经验的列表

        self.buffer_size = buffer_size # 最大容量

    def __len__(self):

        return len(self.buffer) # 返回当前存储的经验数量
```

```
    def push(self, obs, action, reward, next_obs, done): # 存储一条经验

        obs = np.array(obs, dtype=np.float32) # 转换当前状态为 float32 数组

        next_obs = np.array(next_obs, dtype=np.float32) # 转换下一个状态为 float32 数组

        self.buffer.append((obs, action, reward, next_obs, done)) # 添加到经验池

        if len(self.buffer) > self.buffer_size: # 如果超出容量

            self.buffer.pop(0) # 移除最早的经验
```

```
    def sample(self, batch_size): # 随机采样一批经验

        indices = np.random.choice(len(self.buffer), batch_size, replace=False) # 随机选取索引

        batch = [self.buffer[idx] for idx in indices] # 根据索引取出经验

        # 用 np.stack 保证输出为合适的 float/int 数组

        return [np.stack(items) for items in zip(*batch)] # 分别堆叠每一项，返回批量数据
```

```
    def clean(self): # 清空经验池

        self.buffer = []
```



4.3 AgentDQN 类

```
class AgentDQN(Agent):
    def __init__(self, env, args):
        super(AgentDQN, self).__init__(env) # 调用父类 Agent 的构造函数，保存环境
        self.args = args # 保存参数
        obs_dim = env.observation_space.shape[0] # 状态空间维度
        act_dim = env.action_space.n # 动作空间维度
        hidden_dim = 128 # 隐藏层维度
        self.q_net = QNetwork(obs_dim, hidden_dim, act_dim).to(torch.device("cuda" if torch.cuda.is_available()
else "cpu")) # 主 Q 网络
        self.target_q_net = copy.deepcopy(self.q_net) # 目标 Q 网络
        self.optimizer = optim.Adam(self.q_net.parameters(), lr=1e-3) # 主网络的优化器
        self.buffer = ReplayBuffer(buffer_size=10000) # 经验回放池
        self.gamma = 0.99 # 折扣因子
        self.batch_size = 64 # 批量大小
        self.epsilon = 1.0 # 探索率
        self.epsilon_min = 0.01 # 最小探索率
        self.epsilon_decay = 0.995 # 探索率衰减
        self.update_target_steps = 100 # 目标网络更新频率
        self.learn_step_counter = 0 # 学习步数计数
        self.num_episodes = 200 # 训练回合数
```

```
def init_game_setting(self):
    self.epsilon = 1.0 # 每局游戏开始时重置探索率
```

```
def train(self):
    if len(self.buffer) < self.batch_size:
        return
    obs, action, reward, next_obs, done = self.buffer.sample(self.batch_size) # 从经验回放池中采样一个 batch
    # 将采样经验转为 tensor 便于运算
    obs = torch.FloatTensor(obs)
    action = torch.LongTensor(action)
    reward = torch.FloatTensor(reward)
    next_obs = torch.FloatTensor(next_obs)
    done = torch.FloatTensor(done)
```

```
q_values = self.q_net(obs).gather(1, action.unsqueeze(1)).squeeze(1) # 当前的 Q 值
next_q_values = self.target_q_net(next_obs).max(1)[0] # 取 max 下一状态的 Q 值
expected_q = reward + self.gamma * next_q_values * (1 - done) # target 期望的 Q 值
```

```
loss = nn.MSELoss()(q_values, expected_q.detach()) # loss 函数
self.optimizer.zero_grad() # 梯度清零
loss.backward() # 反向传播
self.optimizer.step() # 更新参数
```



```
self.learn_step_counter += 1 # 学习步数加一

if self.learn_step_counter % self.update_target_steps == 0: # 定期同步目标网络
    self.target_q_net.load_state_dict(self.q_net.state_dict())
```

```
def make_action(self, observation, test=True): # 根据当前状态选择动作

    if (not test) and (random.random() < self.epsilon): # 训练时按 epsilon-greedy 策略探索
        return self.env.action_space.sample()

    obs = torch.FloatTensor(observation).unsqueeze(0) # 状态转为 Tensor 并增加 batch 维
    q_values = self.q_net(obs) # 计算所有动作的 Q 值
    return q_values.argmax().item() # 选择 Q 值最大的动作
```

```
def run(self):

    reward_list=[] #用于保存每回合的奖励

    for episode in range(self.num_episodes):

        obs,_ = self.env.reset() # 重置环境, 获得初始状态 # 新版 Gym 返回 (obs, info)
        total_reward = 0 # 本回合累计奖励
        done = False # 回合是否结束

        while not done:

            action = self.make_action(obs, test=False)#选择下一个动作

            next_obs, reward, terminated, truncated, _ = self.env.step(action) #进行下一个动作, 返回值位置、
            # 奖励、是否结束、额外信息 # 新版 Gym 返回 (obs, reward, terminated, truncated, info)

            done = terminated or truncated #terminated 表示游戏结束, truncated 表示时间步数超过限制
            self.buffer.push(obs, action, reward, next_obs, float(done)) #存储经验: 当前状态, 采取动作, 奖励
            # 值, 下一个动作

            obs = next_obs # 状态更新
            total_reward += reward # 累计奖励
            self.train() # 训练网络

        self.epsilon = max(self.epsilon * self.epsilon_decay, self.epsilon_min) # 衰减探索率
        self.target_q_net.load_state_dict(self.q_net.state_dict()) # 每回合同步目标网络
        reward_list.append(total_reward)

        print(f"Episode {episode}, Reward: {total_reward}, Epsilon: {self.epsilon:.3f}") # 打印本回合信息

    # 训练结束后绘制并保存奖励曲线为 SVG

    plt.figure()

    plt.plot(reward_list)

    plt.xlabel('Episode')

    plt.ylabel('Reward')

    plt.title('DQN Reward Curve')

    plt.savefig('reward_curve.svg', format='svg')

    print("奖励曲线已保存为 reward_curve.svg, 请用浏览器打开该文件查看。")
```



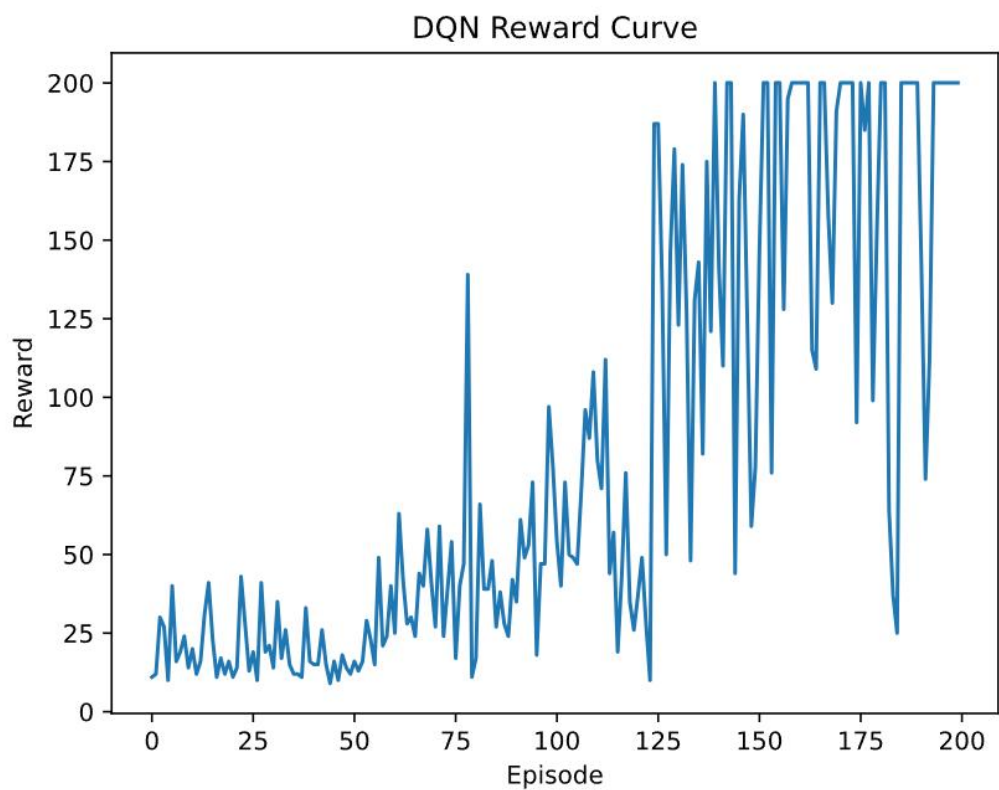
三、 实验结果及分析

1. 实验结果展示示例

1.1 输出每一代 episode 的 reward 值结果

```
Episode 0, Reward: 11.0  
Episode 1, Reward: 12.0  
Episode 2, Reward: 30.0  
Episode 3, Reward: 27.0  
...  
Episode 197, Reward: 200.0  
Episode 198, Reward: 200.0  
Episode 199, Reward: 200.0
```

1.2 可视化的 Reward 曲线图



2. 评测指标展示及分析

2.1 Reward 大小

从终端输出结果可以看出，DQN 智能体在训练初期 reward 波动较大，reward 值大多在 10~60 之间，偶尔有较高分数。随着训练进行，reward 逐渐增大，reward=200（为环境最大分数）出现的频率越来越高。

大约在第 120~150 回合后，reward 开始频繁达到 200，且后续回合 reward 基本稳定在 200，说明智能体已经学会了最优策略。

2.2 收敛性分析

前期（0~100 回合）：reward 波动大，未收敛。

中期（100~150 回合）：reward 快速提升，reward=200 出现频率增加。

后期（150 回合以后）：reward 基本稳定在 200，算法收敛。

四、 优化

1. 优化一：DoubleDQN 算法

1.1 原理

普通 DQN 用目标网络直接对 next_obs 取最大 Q 值。

Double DQN 用主网络选动作（argmax），再用目标网络评估该动作的 Q 值

1.2 代码

```
"""普通 DQN 算法"""

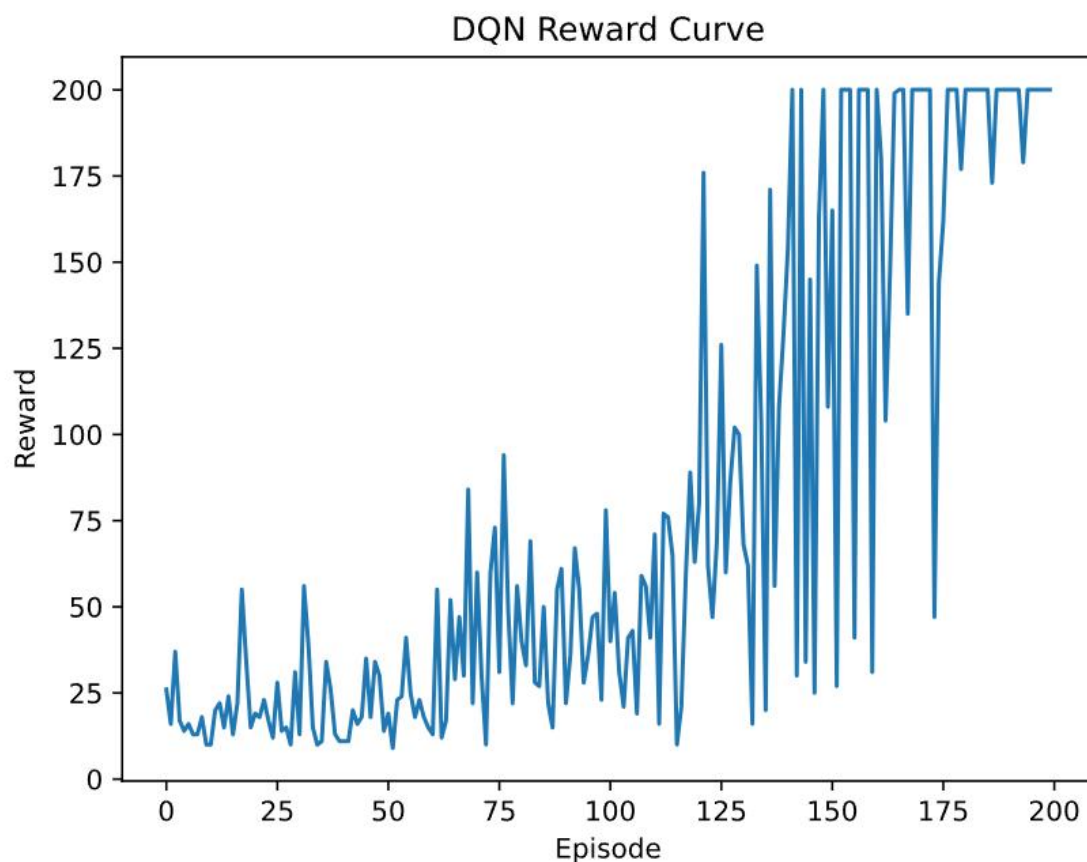
# next_q_values = self.target_q_net(next_obs).max(1)[0] #取 max 下一状态的 Q 值
# expected_q = reward + self.gamma * next_q_values * (1 - done) #target 期望的 Q 值

"""Double DQN: 用主网络选动作，用目标网络评估 Q 值"""

next_actions = self.q_net(next_obs).argmax(1, keepdim=True) # 主网络选动作
next_q_values = self.target_q_net(next_obs).gather(1, next_actions).squeeze(1) # 目标网络评估 Q 值
expected_q = reward + self.gamma * next_q_values * (1 - done)
```




1.3 优化结果展示



1.4 分析

优点：Double DQN 的收敛较为提前，且收敛结果更稳定

缺点：前期 reward 变化较为保守

2. 优化二：QNetwork 正则化+软更新

2.1 原理

用软更新替代硬同步，提升后期稳定性。

增加一层隐藏层（如三层全连接），提升表达能力。

在每层后加 `nn.LayerNorm` 或 `nn.Dropout(0.1)`，提升泛化和稳定性。

2.2 代码

```
# QNetwork 增加正则化
class QNetwork(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(QNetwork, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.norm1 = nn.LayerNorm(hidden_size)
        self.fc2 = nn.Linear(hidden_size, hidden_size)
        self.norm2 = nn.LayerNorm(hidden_size)
        self.fc3 = nn.Linear(hidden_size, output_size)

    def forward(self, x):
```



```
x = torch.relu(self.norm1(self.fc1(x)))  
x = torch.relu(self.norm2(self.fc2(x)))  
x = self.fc3(x)  
return x
```

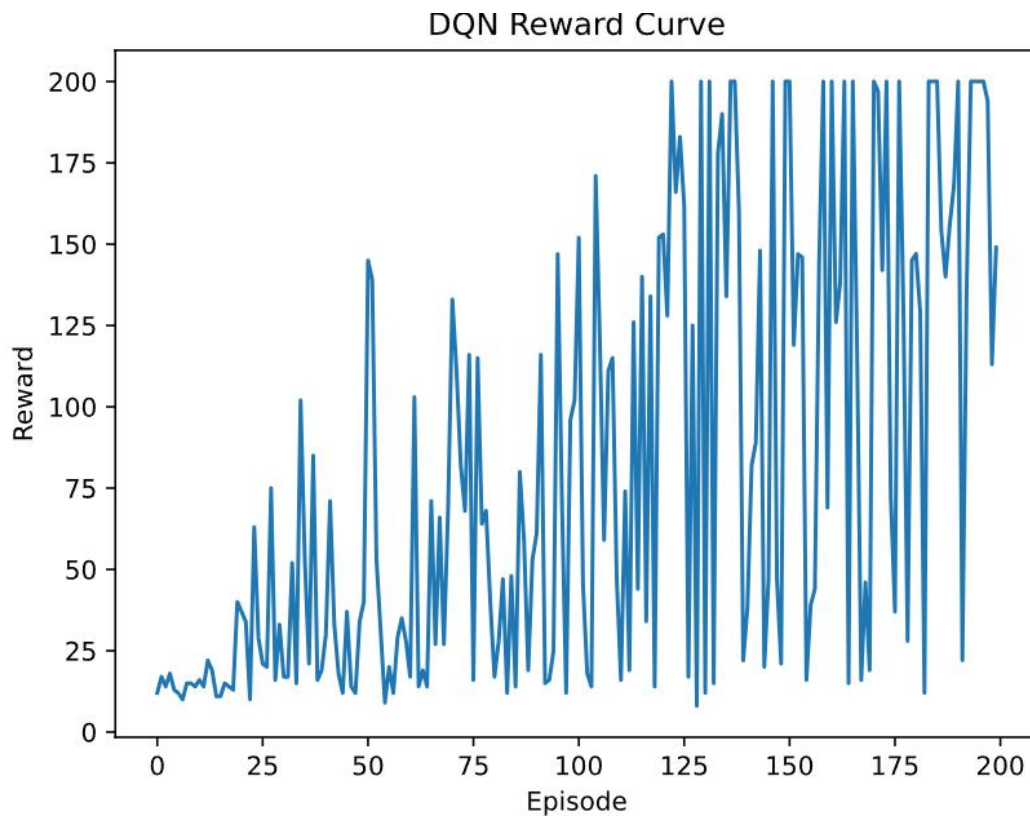
软更新

```
def soft_update(self):  
    for target_param, param in zip(self.target_q_net.parameters(), self.q_net.parameters()):  
        target_param.data.copy_(self.tau * param.data + (1.0 - self.tau) * target_param.data)
```

在 train() 最后调用

```
self.soft_update()
```

2.3 结果展示及分析



提升效果不显著，不做多余赘述



3. 优化三：调整参数

3.1 原理

hidden_dim 增大到 256，提升网络拟合能力。

lr（学习率）增大到 $5e-4$ ，加快参数更新。

buffer_size 增大到 50000，防止经验过早被覆盖。

gamma 略微降低到 0.98，提升短期奖励权重，加快收敛。

batch_size 增大到 128，提升训练稳定性。

epsilon_min 提高到 0.05，保证后期仍有一定探索。

epsilon_decay 提高到 0.990，让智能体更快进入利用阶段。

update_target_steps 降低到 50，更频繁同步目标网络，提升训练稳定性。

3.2 代码

```
hidden_dim = 256 # 隐藏层维度

self.q_net = QNetwork(obs_dim, hidden_dim, act_dim).to(torch.device("cuda" if torch.cuda.is_available()
else "cpu")) # 主 Q 网络

self.target_q_net = copy.deepcopy(self.q_net) # 目标 Q 网络

self.optimizer = optim.Adam(self.q_net.parameters(), lr=5e-4) # 主网络的优化器

self.buffer = ReplayBuffer(buffer_size=50000) # 经验回放池

self.gamma = 0.98 # 折扣因子

self.batch_size = 128 # 批量大小

self.epsilon = 1.0 # 探索率

self.epsilon_min = 0.05 # 最小探索率

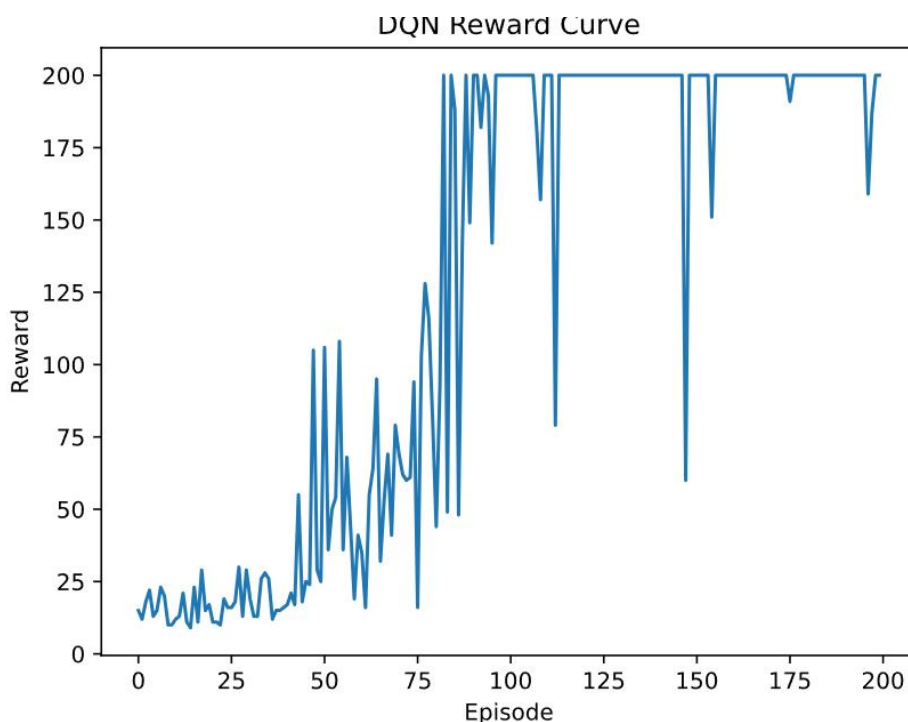
self.epsilon_decay = 0.99 # 探索率衰减

self.update_target_steps = 50 # 目标网络更新频率

self.learn_step_counter = 0 # 学习步数计数

self.num_episodes = 200 # 训练回合数
```

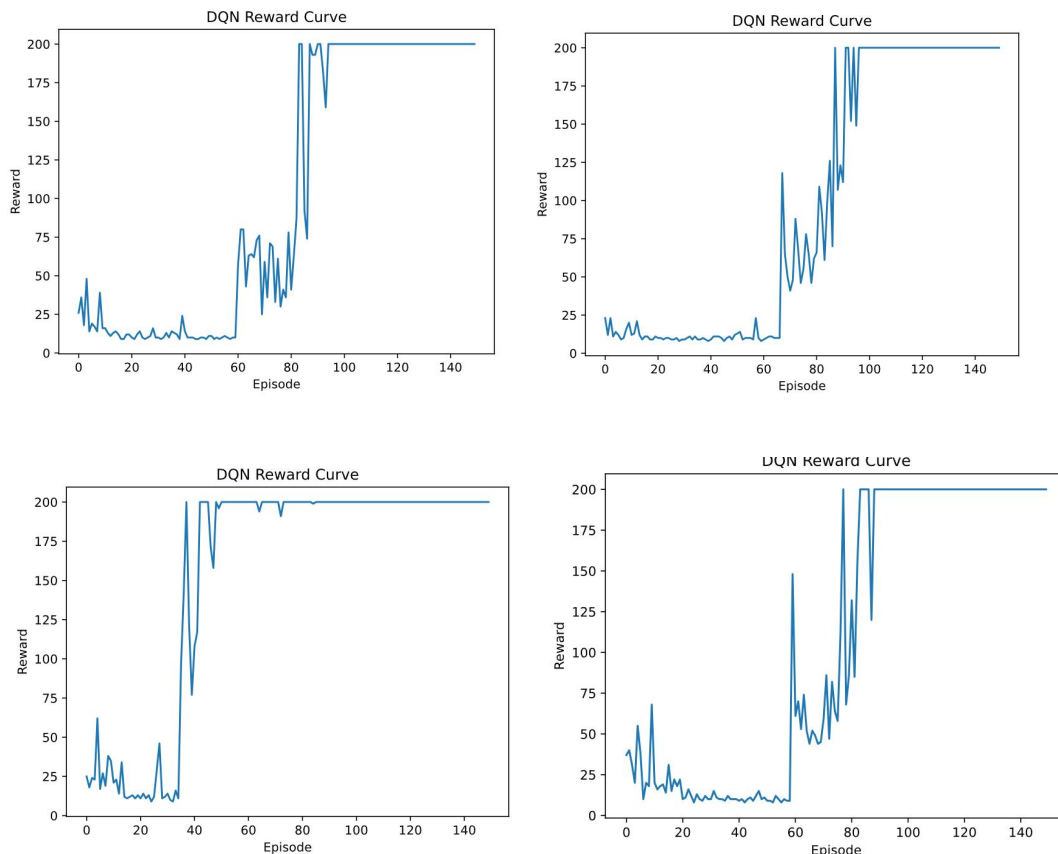
3.3 优化结果展示



3.4 分析

算法效果显著提升，reward 很快就达到了收敛的效果

3.5 接着进行超参数的微调整得到下列较为不错的结果



五、 思考

1. DQN 收敛速度与稳定性的影响因素

实验过程中发现，DQN 的收敛速度和最终 reward 的稳定性受多种因素影响，包括网络结构、超参数设置、目标网络的更新方式等。合理的参数设置（如较大的隐藏层、适中的学习率、较大的经验池和 batch size）可以显著加快收敛速度，使 reward 更早达到 200，并且收敛后表现更加稳定。

2. 优化方法的实际效果

Double DQN 有效缓解了 Q 值高估问题，使得 reward 收敛更快且波动更小。网络结构正则化和软更新虽然提升有限，但在部分环境下有助于提升泛化能力和后期表现。参数调整对收敛速度和最终 reward 影响最大，合理的参数组合能让算法在更少的 episode 内达到最优表现。

3. 进一步提升的方向

进一步提升 DQN 的性能，可以考虑引入优先经验回放、多步 TD、Dueling DQN 等进阶方法。此外，reward 归一化、奖励裁剪、探索策略改进等也有助于提升收敛速度和稳定性。实际应用中还需根据环境特性不断调参和实验，找到最适合当前任务的 DQN 变体和参数组合。



六、 参考资料

1. https://blog.csdn.net/qq_42978535/article/details/144733948
2. <https://blog.csdn.net/niulinbiao/article/details/133690617>
3. BV1Za4y1d7YJ